Event Management Service Coordination System

1. Introduction

The Event Management Service Coordination System is designed to support an event manager in organizing and coordinating multiple service teams for events such as weddings, corporate gatherings, and private parties. The system enables the event manager to create event plans, assign tasks to various teams, track preparation progress, and send automated reminders and alerts to ensure smooth execution leading up to the event day.

The solution provides a structured, automated workflow for communication between the event manager and service teams, reducing manual coordination effort, improving event timelines, and ensuring all teams stay aligned with expectations.

2. Problem Statement

Event coordination often involves numerous teams and strict timelines. Manual communication can lead to delays, overlooked tasks, poor coordination, and last-minute issues. To address these challenges, the event manager needs a system that can:

- Create and manage event details (type, date, expected guests).
- Assign tasks to service teams with clear instructions and deadlines.
- Send automated notifications and preparation reminders.
- Request periodic task status updates.
- Immediately alert the event manager if issues or delays arise.
- Send final event-day alerts to ensure all teams are prepared.

This system aims to simplify team communication, improve tracking, and enable proactive issue handling.

3. Architecture Overview

The system follows a high-level microservice-based architecture that supports modularity, scalability, and asynchronous communication. The major components include:

Frontend / Application Layer

Interface for the event manager to create events, assign tasks, track progress, and handle issues.

API Layer (Play Framework)

Provides REST endpoints for event creation, task assignment, and progress tracking.

Service Layer

Handles business logic:

- Event creation
- Task assignment workflows
- Timeline-based notifications and reminders
- Issue escalation logic

Database Layer (MySQL)

Stores persistent data, including:

- Event details
- Task assignments
- Team information
- Task statuses
- Notifications log

Notification Service (Akka Actors)

Manages automated reminders and real-time alerts using:

- Scheduled reminders
- Progress update triggers

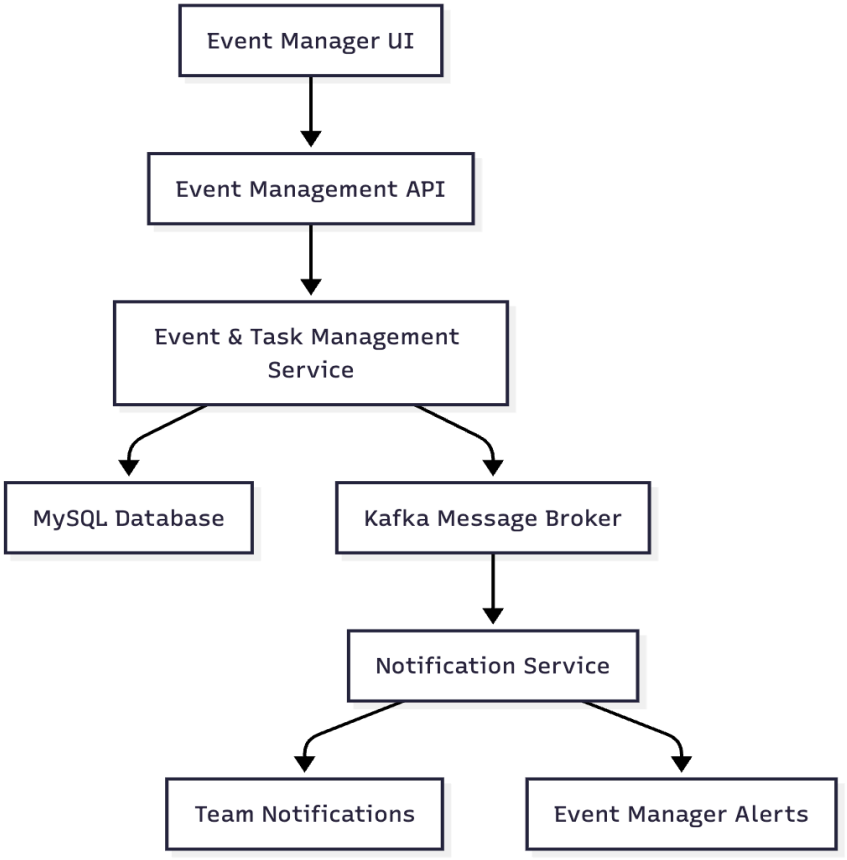
- Issue alerts
- Event-day notifications

Message Broker (Kafka)

Used for reliable asynchronous communication between the main application and Notification Service.

Deployment Layer (Docker)

Offers consistent, portable, and scalable deployment across all environments.



4. Component Design

Frontend (Event Manager Interface)

- Create event plans.
- Assign tasks to teams.
- Provide instructions & deadlines.
- View progress status.
- Receive issue alerts.

Play API Controllers

- Accept REST API requests from frontend.
- Validate requests.
- Forward data to Service Layer.

Service Layer

- Apply business rules for event creation and task assignment.
- Publish task-related events to Kafka (task assigned, issue raised, reminder needed).
- Coordinate with Notification Service for reminders and alerts.

Database (MySQL)

Includes the following structured tables:

- **Events**: Event type, date, guest count.
- **Tasks**: Specific team assignments.
- **Teams**: Catering, Decorations, Entertainment, Logistics.
- **Reminders**: Scheduled events and logs.
- **Issue Reports**: From teams to event manager.

Notification Service (Akka)

- Uses actors for concurrent notification processing.
- Schedulers trigger:
    - Task assignment notifications
    - Preparation reminders (1 day before + few hours before)
    - Progress check-ins

    - Event-day alerts
- Detects issue events from Kafka and notifies event manager immediately.

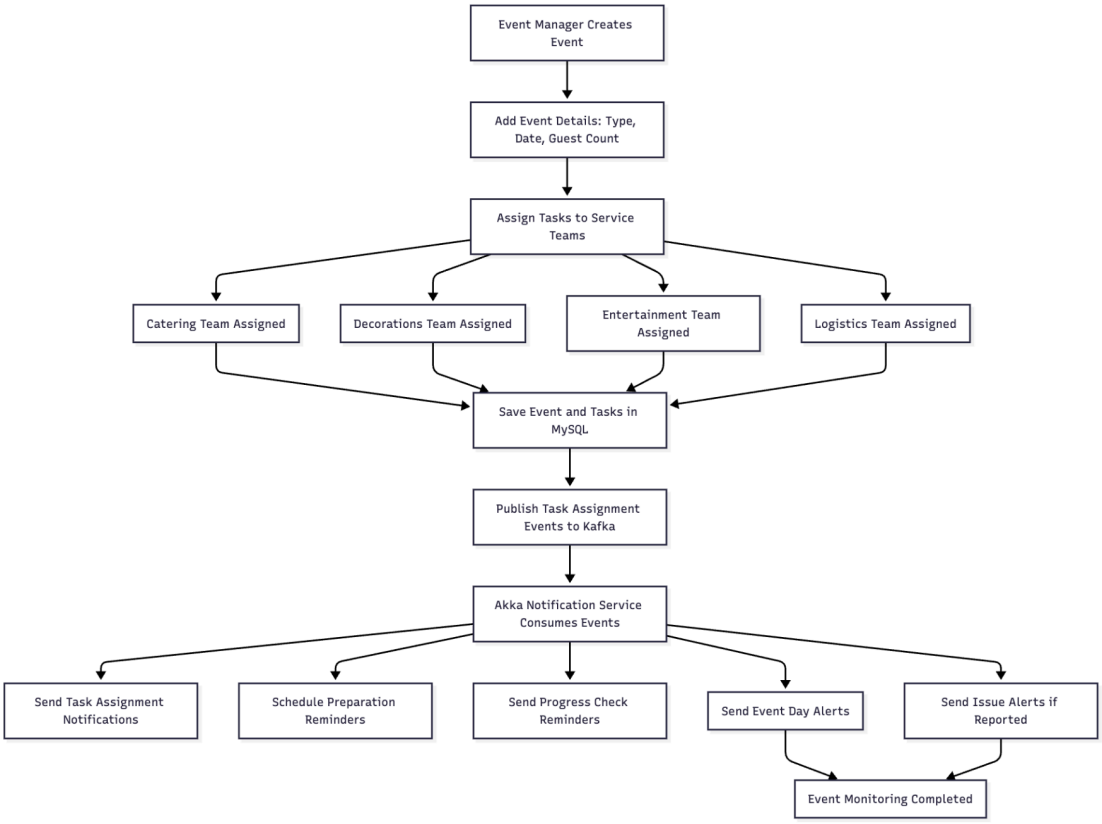
Kafka Messaging Layer

- Ensures reliable, asynchronous delivery of notification events.
- Guarantees no loss of reminders or alerts.

Docker Deployment

- Each service packaged as a separate container.
- Allows independent scaling (e.g., Notification Service under heavy load).

5. System Workflow



6. Microservices Design

1. Event Management Service

Purpose:
Manages all event operations:

- Event creation
- Task assignment
- Progress update handling
- Issue reporting

Responsibilities:

- Hosting all REST APIs
- Coordinating event workflows
- Publishing task-related events to Kafka

2. Notification Service

Purpose:
Handles all automated reminders and alerts.

Responsibilities:

- Consume Kafka events
- Send task assignment notifications
- Send preparation reminders
- Send progress check-in reminders
- Issue escalation alerts
- Final event-day alerts

Inter-Service Communication

- **Synchronous:**
  Frontend → Event Management Service (REST)
- **Asynchronous:**
  Event Management Service → Notification Service (Kafka)

7.Data Base Design

CREATE TABLE event_users (id BIGINT AUTO_INCREMENT PRIMARY KEY, username VARCHAR(50) UNIQUE, password VARCHAR(255), user_role VARCHAR(20), email VARCHAR(100), created_at TIMESTAMP, updated_at TIMESTAMP);

CREATE TABLE teams (id BIGINT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50) UNIQUE, created_at TIMESTAMP, updated_at TIMESTAMP);

CREATE TABLE events (id BIGINT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100), type VARCHAR(20), date TIMESTAMP, guest_count INT, created_by BIGINT, created_at TIMESTAMP, updated_at TIMESTAMP, UNIQUE KEY unique_event (name, date), FOREIGN KEY (created_by) REFERENCES event_users(id));

CREATE TABLE team_users (id BIGINT AUTO_INCREMENT PRIMARY KEY, team_id BIGINT, user_id BIGINT, FOREIGN KEY (team_id) REFERENCES teams(id), FOREIGN KEY (user_id) REFERENCES event_users(id));

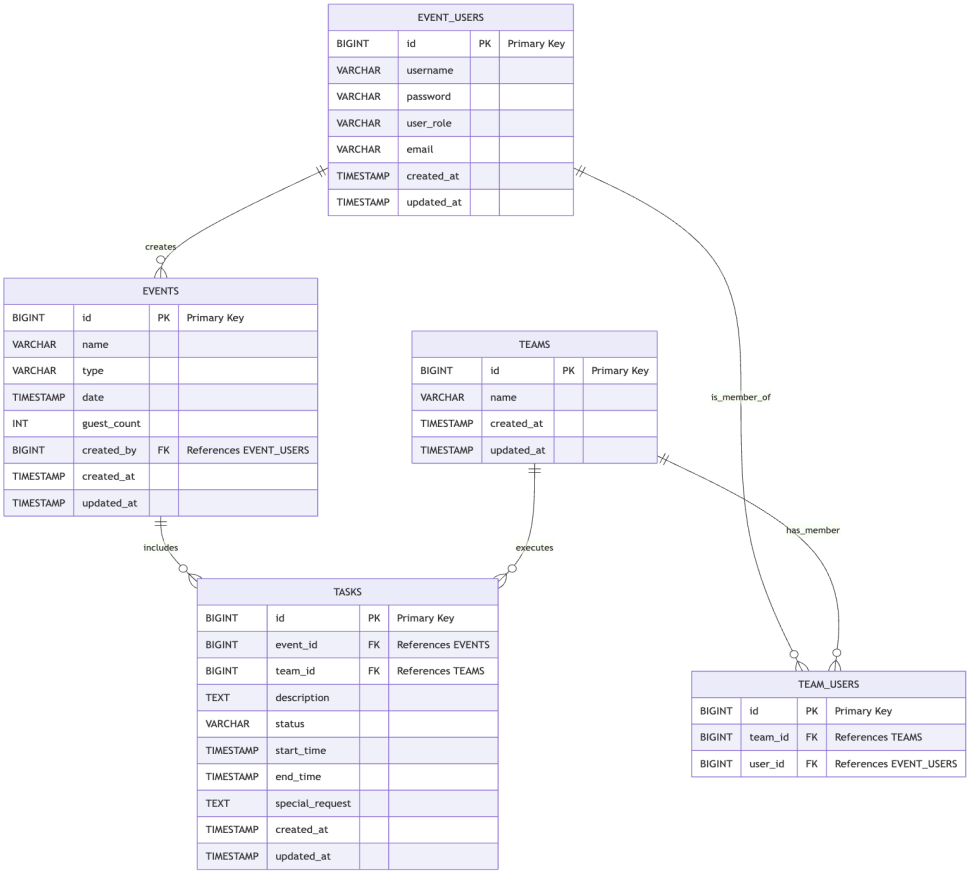
CREATE TABLE tasks (id BIGINT AUTO_INCREMENT PRIMARY KEY, event_id BIGINT, team_id BIGINT, description TEXT, status VARCHAR(20), start_time TIMESTAMP, end_time TIMESTAMP, special_request TEXT, created_at TIMESTAMP, updated_at TIMESTAMP, UNIQUE KEY unique_task (event_id, team_id, description(255)), FOREIGN KEY (event_id) REFERENCES events(id) ON DELETE CASCADE, FOREIGN KEY (team_id) REFERENCES teams(id));

INSERT INTO event_users (id, username, email, password, user_role, created_at, updated_at) VALUES
(1,'shekar','naresh996463@gmail.com','Shekar@9705','EVENT_MANAGER','2025-11-20 17:11:48','2025-11-20 17:11:48'),(6,'jane.doe','naresh119935@gmail.com','SecurePass123!','TEAM_MEMBER','2025-11-21 11:54:11','2025-11-21 11:54:11'),(7,'rohith','nareshnagula61@gmail.com','Rohith@123!','TEAM_MEMBER','2025-11-21 23:54:20','2025-11-21 23:54:20');

INSERT INTO teams (id, name, created_at, updated_at) VALUES (5,'Catering Team','2025-11-21 18:39:21','2025-11-21 18:39:21'),(6,'Decorations Team','2025-11-21 23:43:06','2025-11-21 23:43:06');
INSERT INTO events (id, name, type, date, guest_count, created_by, created_at, updated_at) VALUES (5,'John and Jane Wedding','Wedding','2025-12-10 10:00:00',100,6,'2025-11-21 18:41:35','2025-11-21 18:41:35'),(8,'Sweety','BirthDay','2025-12-10 10:00:00',100,6,'2025-11-21 23:51:05','2025-11-21 23:51:05');
INSERT INTO team_users (id, team_id, user_id) VALUES (6,5,6),(8,5,7),(7,6,7);
INSERT INTO tasks (id, event_id, team_id, description, status, start_time, end_time, special_request, created_at, updated_at) VALUES (1,5,5,'Prepare appetizers and main course','Pending','2025-12-10 08:00:00','2025-12-10 12:00:00','Include vegan options','2025-11-22 10:00:00','2025-11-22 10:00:00'),(2,5,6,'Decorate hall and stage','Pending','2025-12-10 07:00:00','2025-12-10 11:00:00','Use red and white theme','2025-11-22 10:05:00','2025-11-22 10:05:00'),(3,8,5,'Prepare birthday cake and snacks','Pending','2025-12-10 08:30:00','2025-12-10 12:30:00','Chocolate cake with candles','2025-11-22 10:10:00','2025-11-22 10:10:00'),(4,8,6,'Set up balloons and

decorations','Pending','2025-12-10 07:30:00','2025-12-10 11:30:00','Blue and yellow theme','2025-11-22 10:15:00','2025-11-22 10:15:00'),(5,5,5,'Arrange drinks and beverages','Pending','2025-12-10 09:00:00','2025-12-10 12:00:00','Include soft drinks','2025-11-22 10:20:00','2025-11-22 10:20:00'),(6,5,6,'Set up photo booth and props','Pending','2025-12-10 09:00:00','2025-12-10 13:00:00','Include backdrop with couple's name','2025-11-22 10:25:00','2025-11-22 10:25:00'),(7,8,5,'Prepare party favors','Pending','2025-12-10 10:00:00','2025-12-10 13:00:00','Small gift bags for kids','2025-11-22 10:30:00','2025-11-22 10:30:00'),(8,8,6,'Decorate birthday cake table','Pending','2025-12-10 08:30:00','2025-12-10 11:00:00','Add balloons around table','2025-11-22 10:35:00','2025-11-22 10:35:00');

INSERT INTO notifications (id, task_id, type, sent_at, status, created_at, updated_at) VALUES (1,1,'Email',NULL,'Pending','2025-11-22 10:00:00','2025-11-22 10:00:00'),(2,2,'Email',NULL,'Pending','2025-11-22 10:05:00','2025-11-22 10:05:00'),(3,3,'Email',NULL,'Pending','2025-11-22 10:10:00','2025-11-22 10:10:00'),(4,4,'Email',NULL,'Pending','2025-11-22 10:15:00','2025-11-22 10:15:00'),(5,5,'Email',NULL,'Pending','2025-11-22 10:20:00','2025-11-22 10:20:00'),(6,6,'Email',NULL,'Pending','2025-11-22 10:25:00','2025-11-22 10:25:00'),(7,7,'Email',NULL,'Pending','2025-11-22 10:30:00','2025-11-22 10:30:00'),(8,8,'Email',NULL,'Pending','2025-11-22 10:35:00','2025-11-22 10:35:00');



8. REST API Endpoints

Event Management Service API

Authentication

1. Login

URL: POST — http://localhost:9000/api/login

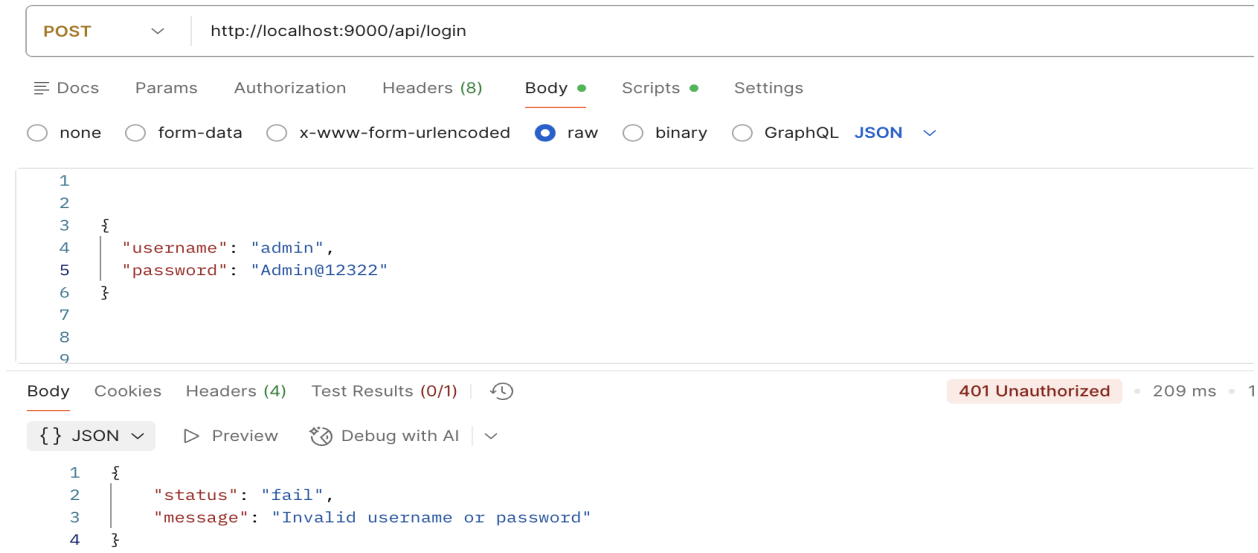
Input:

```
{

 "username": "Shekar",

 "password": "Shekar@9705"

}
```

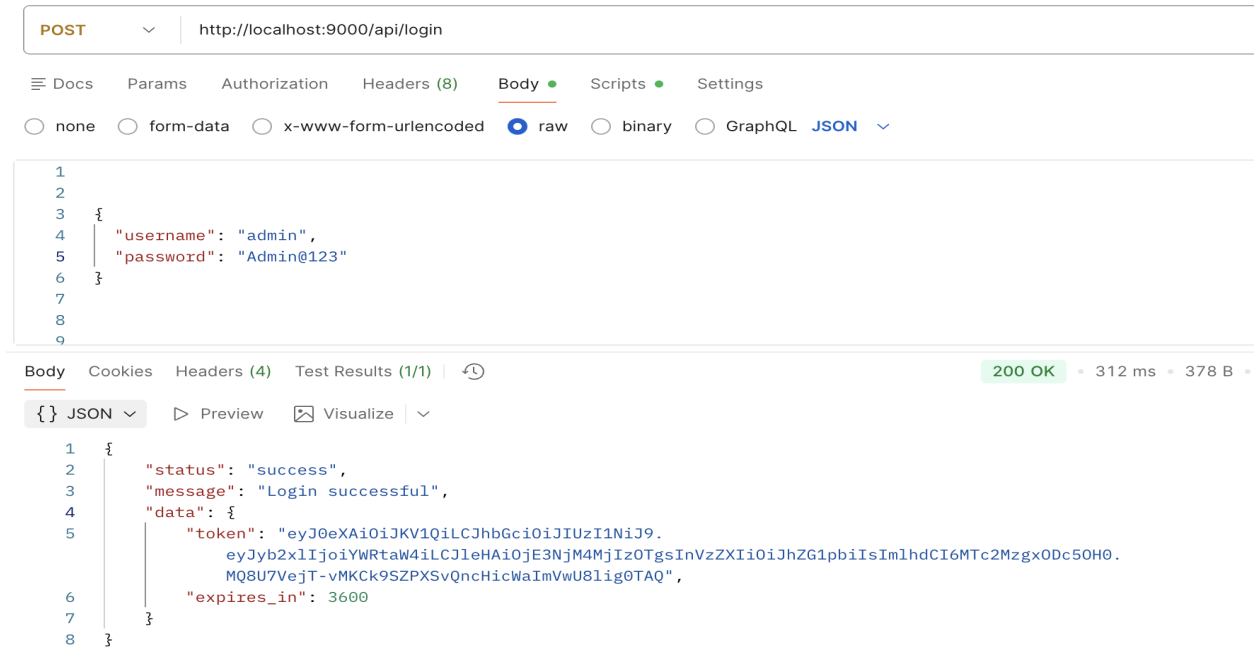
Output:

```
{

   "status": "success",

   "message": "Login successful",

   "data": {

       "token":
"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoiYWRtaW4iLCJleHAiOjE3NjM4MjY0M
zMsInVzZXIiOiJhZG1pbiIsImlhdCI6MTc2MzgyMjgzM30.YwsJ5pb0BVWPtsra7XdgFoju8RJ8iMri
h8_zzPsbiWA",

       "expires_in": 3600

   }

}
```

Case fail:



Case Success:



User Management

Create User

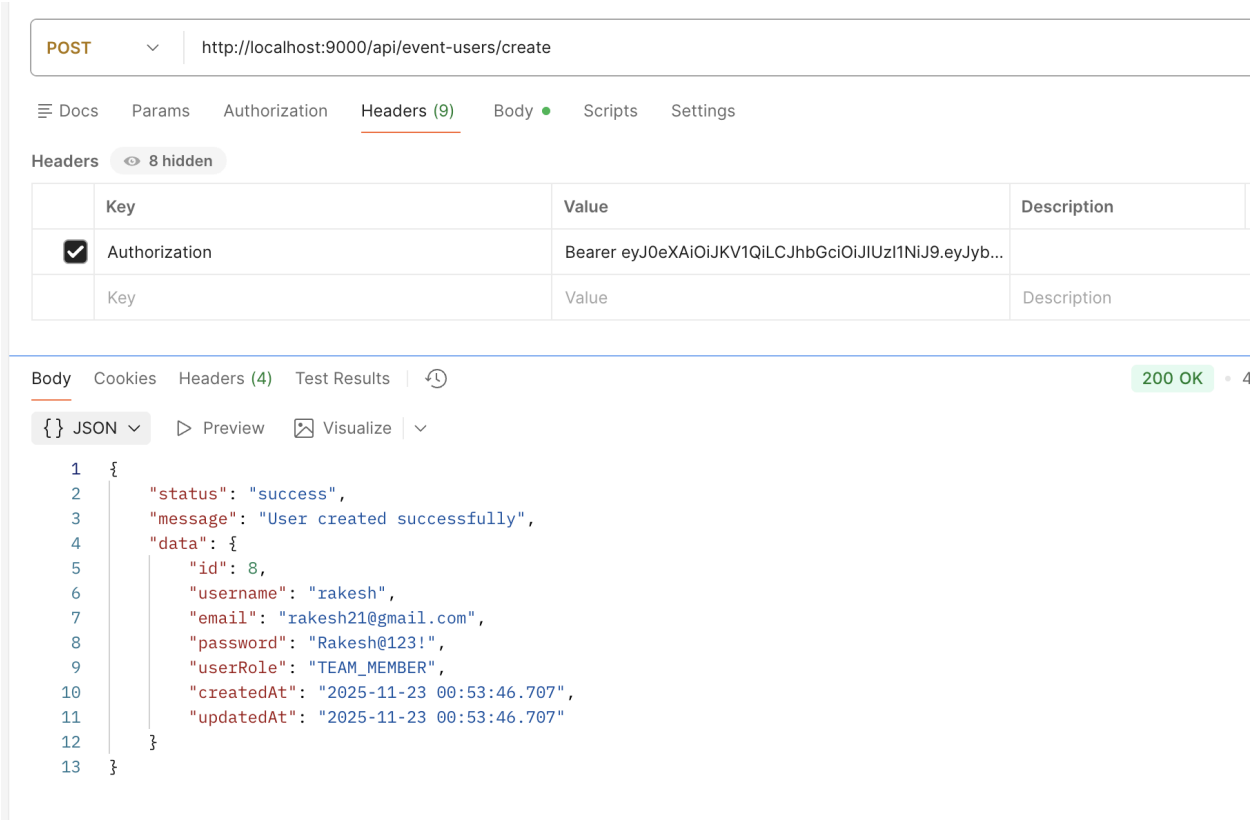
URL: POST http://localhost:9000/api/event-users/create

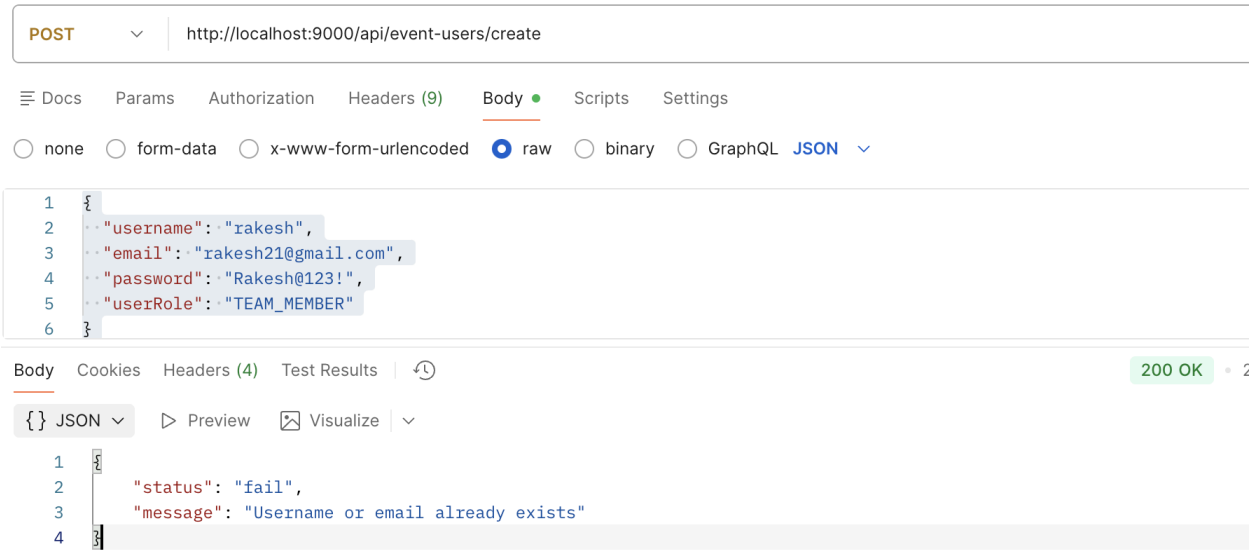
Input:


```
{
 "username": "rakesh",
 "email": "rakesh21@gmail.com",
 "password": "Rakesh@123!",
 "userRole": "TEAM_MEMBER"
}
```


OUTPUTS:

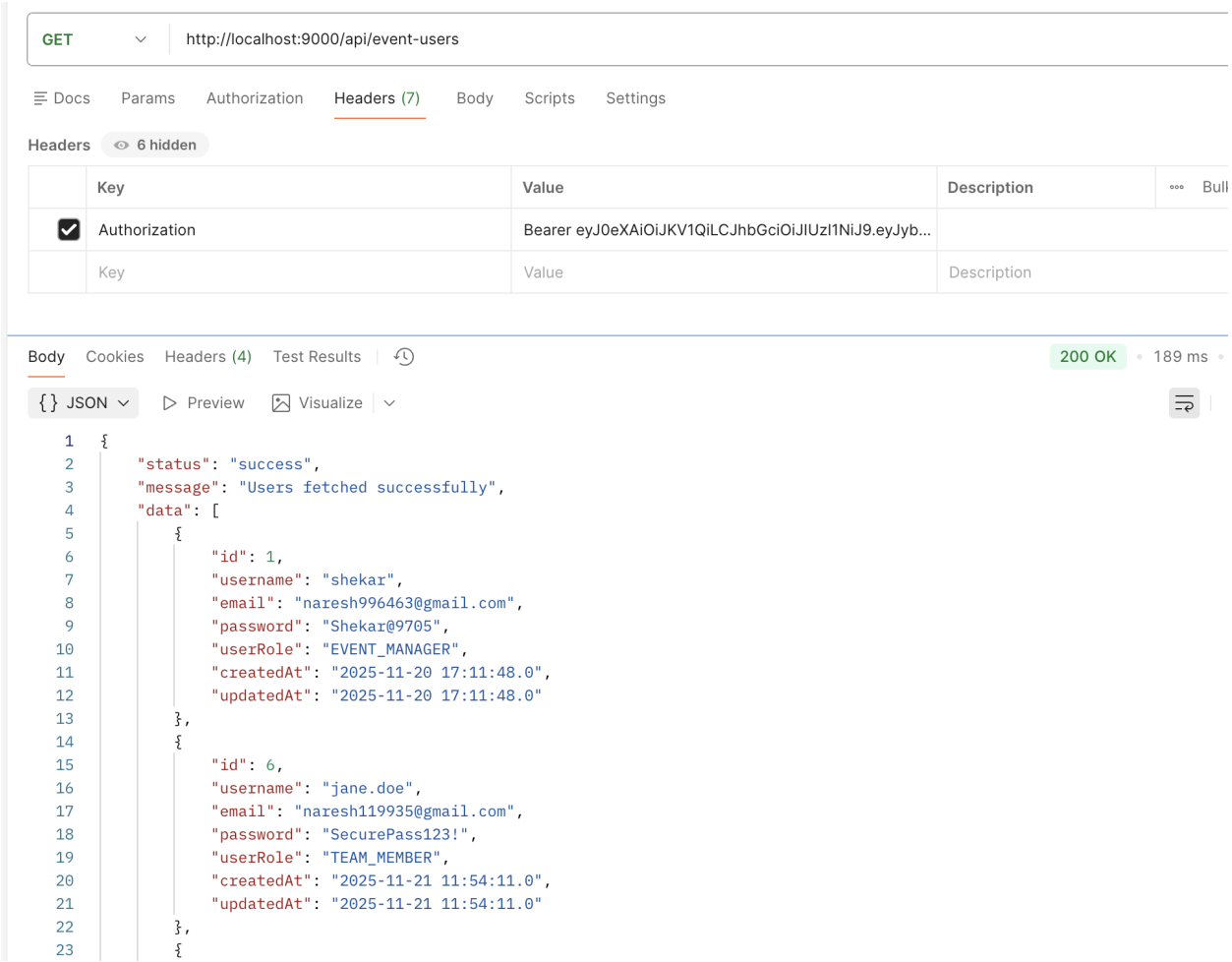

```
{
  "status": "fail",
  "message": "Username or email already exists"
}
{
  "status": "success",
  "message": "User created successfully",
  "data": {
    "id": 8,
    "username": "rakesh",
    "email": "rakesh21@gmail.com",
    "password": "Rakesh@123!",
    "userRole": "TEAM_MEMBER",
    "createdAt": "2025-11-23 00:53:46.707",
    "updatedAt": "2025-11-23 00:53:46.707"
  }
}
```


POST
http://localhost:9000/api/event-users/create
Docs
Params
Authorization
Headers (9)
Body
Scripts
Settings
Headers
8 hidden
Key
Value
Description
Authorization
Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyb...
Key
Value
Description
Body
Cookies
Headers (4)
Test Results
200 OK
JSON
Preview
Visualize
{
"status": "success",
"message": "User created successfully",
"data": {
"id": 8,
"username": "rakesh",
"email": "rakesh21@gmail.com",
"password": "Rakesh@123!",
"userRole": "TEAM_MEMBER",
"createdAt": "2025-11-23 00:53:46.707",
"updatedAt": "2025-11-23 00:53:46.707"
}
}


POST
http://localhost:9000/api/event-users/create
Docs
Params
Authorization
Headers (9)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"username": "rakesh",
"email": "rakesh21@gmail.com",
"password": "Rakesh@123!",
"userRole": "TEAM_MEMBER"
}
Body
Cookies
Headers (4)
Test Results
200 OK
JSON
Preview
Visualize
{
"status": "fail",
"message": "Username or email already exists"
}

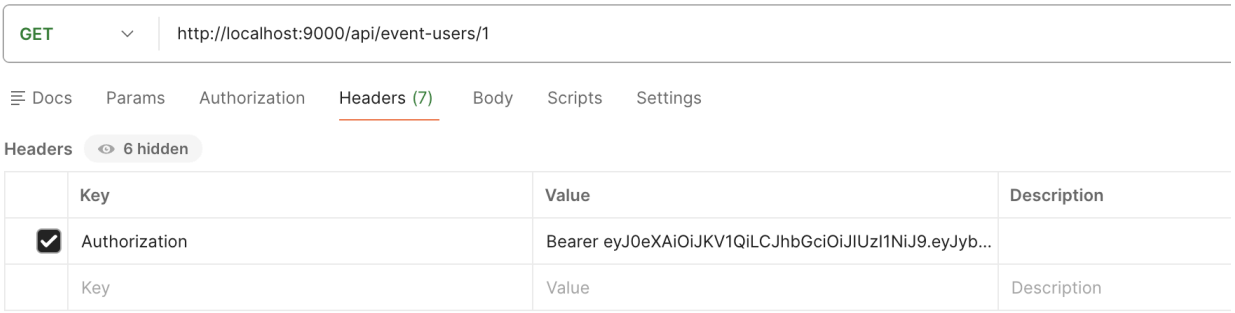
Get All Users

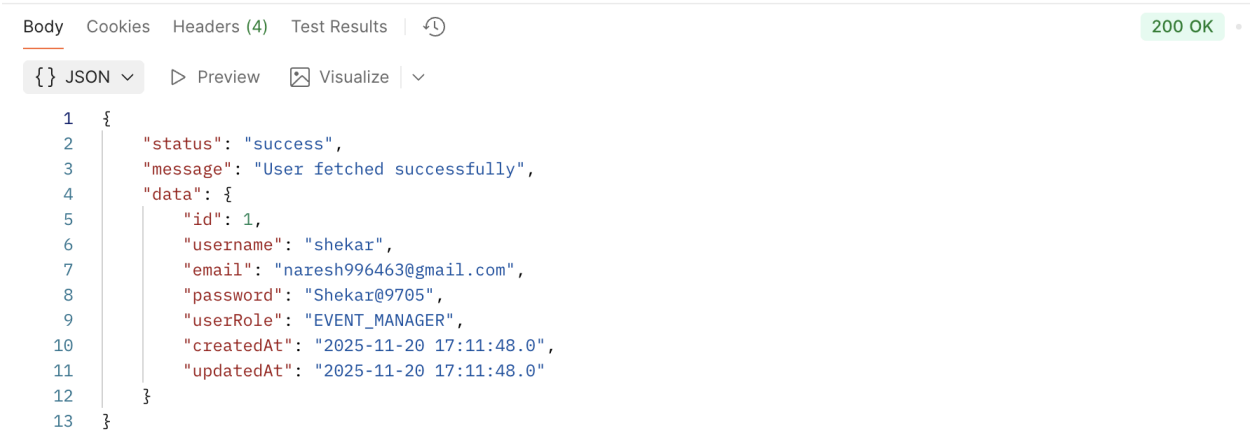
URL: GET http://localhost:9000/api/event-users



Get User by ID

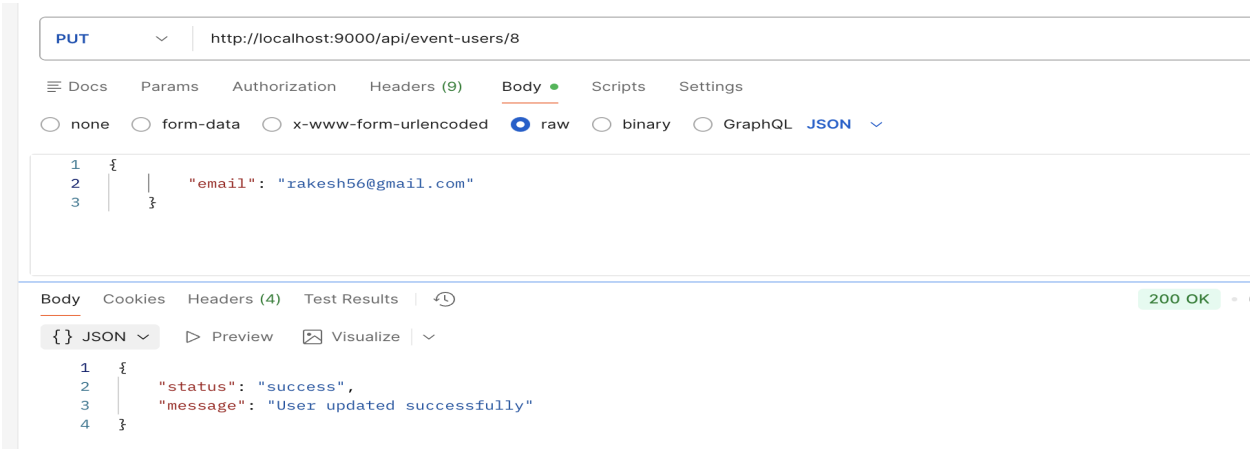
URL: GET http://localhost:9000/api/event-users/1





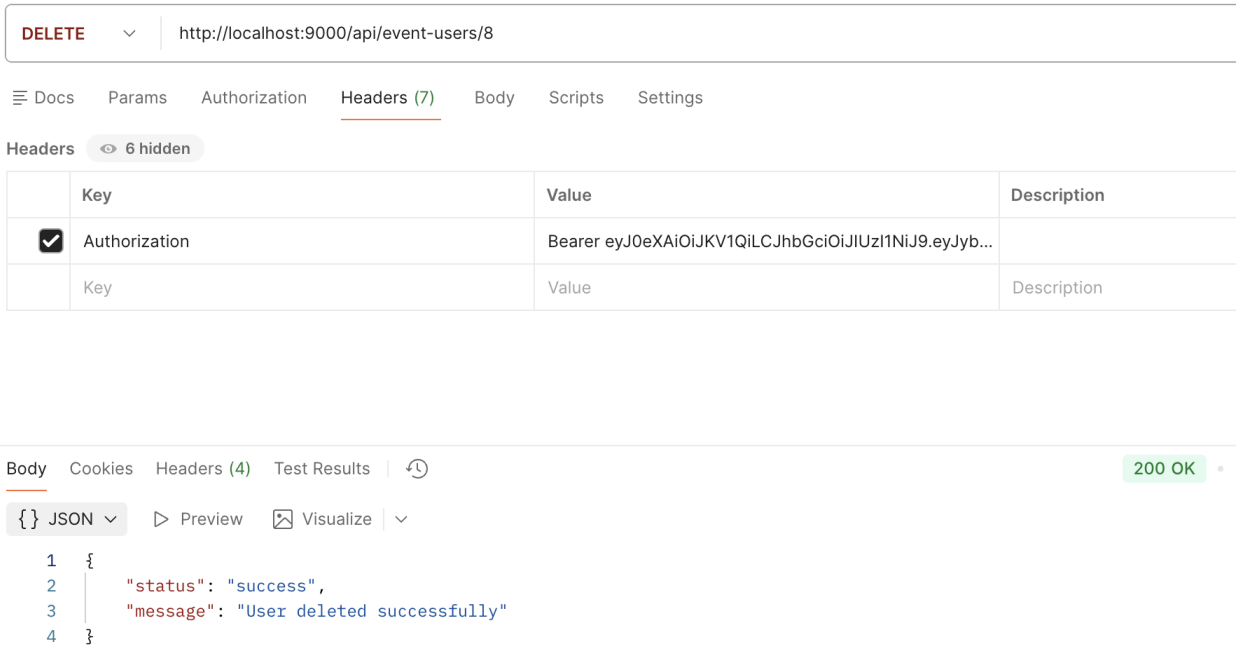
Update User

URL: PUT http://localhost:9000/api/event-users/7



Delete User

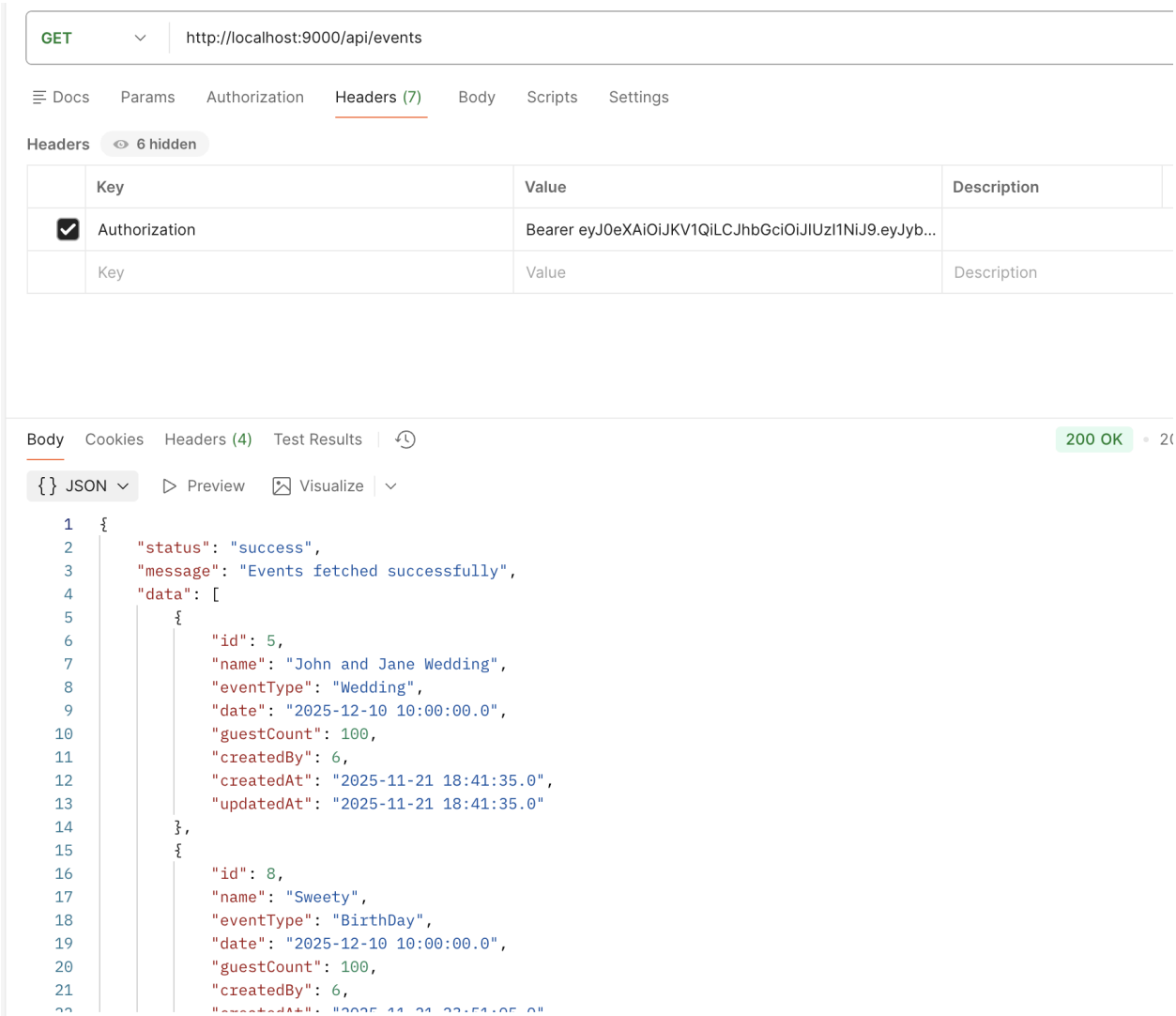
URL: DELETE http://localhost:9000/api/event-users/8



Event Management

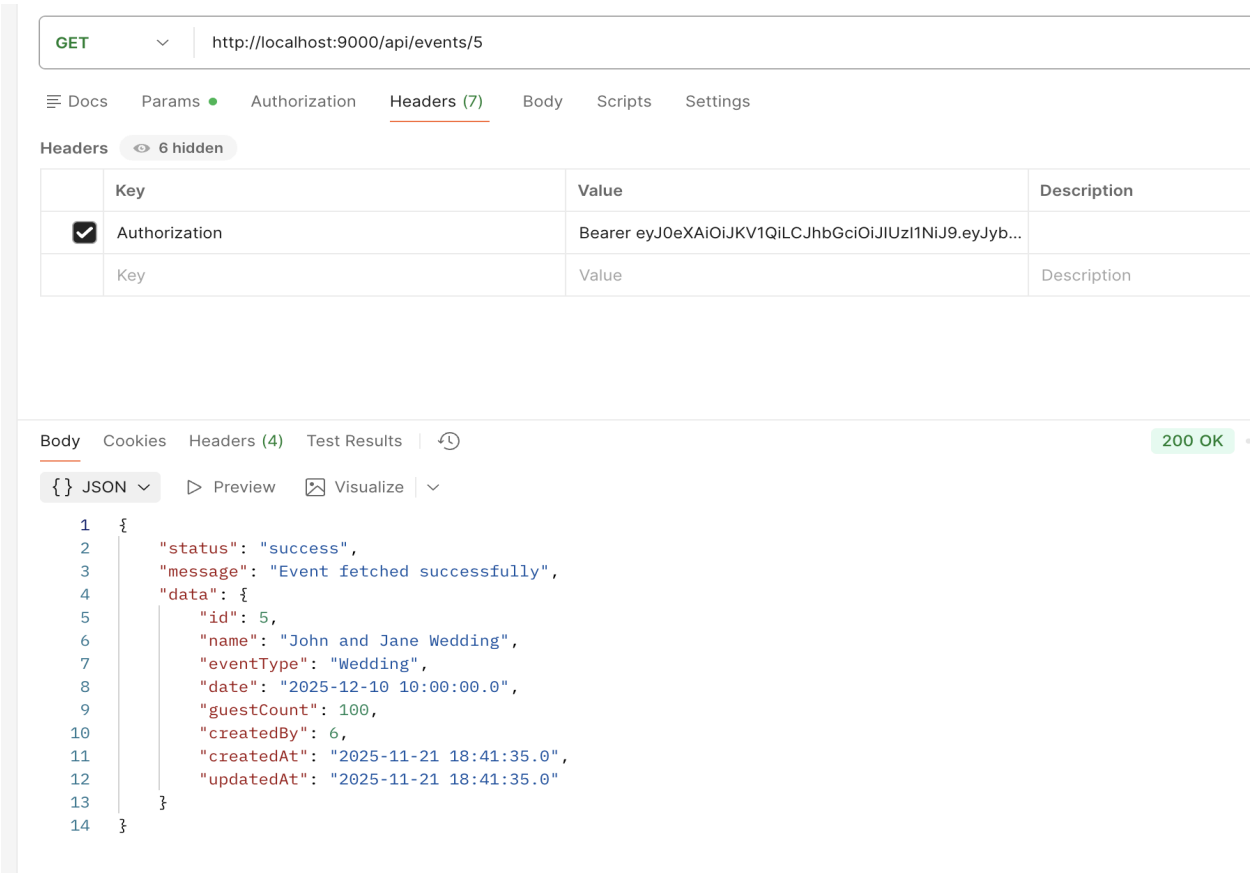
Get All Events

URL: GET http://localhost:9000/api/events



Get Event By ID

URL: GET http://localhost:9000/api/events/5



Create an Event

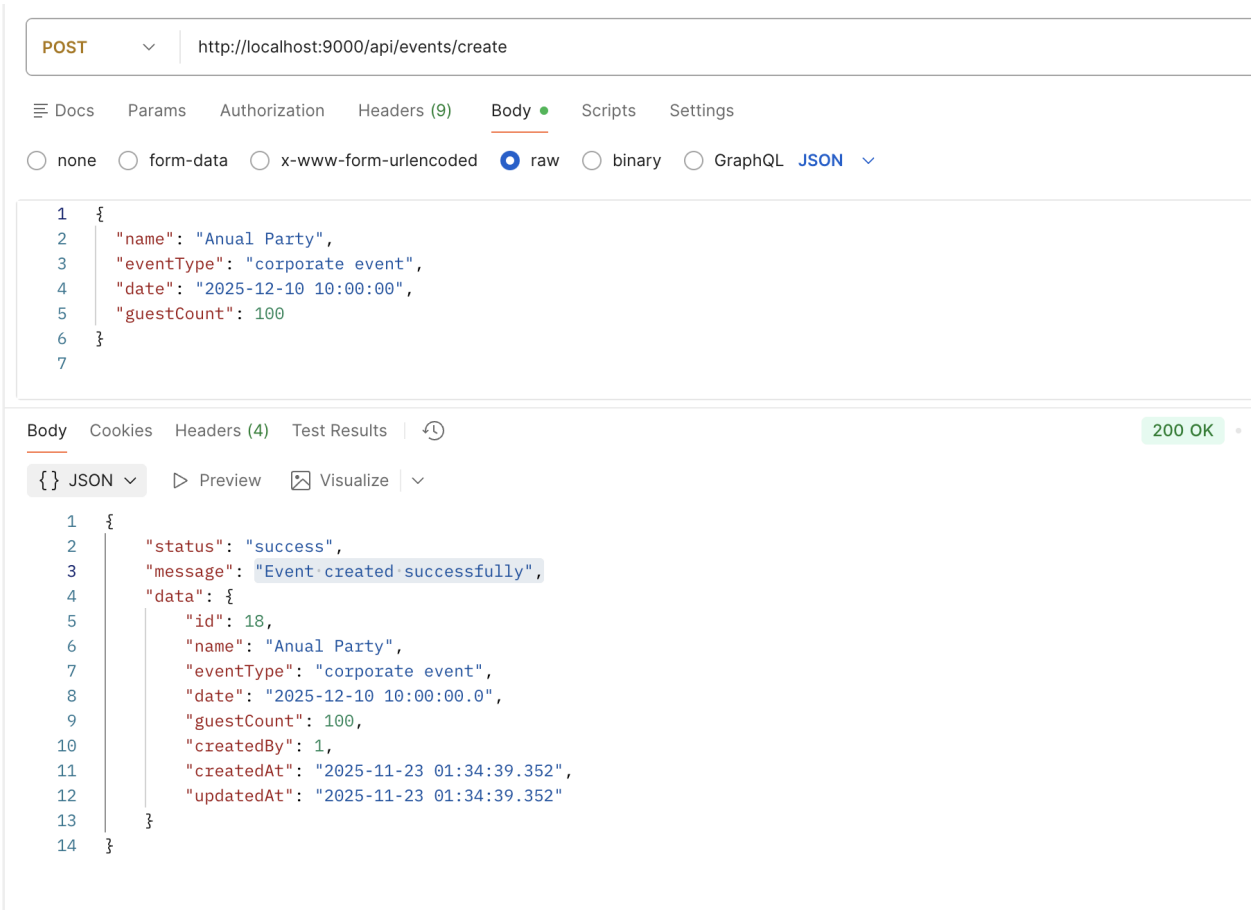
URL: POST http://localhost:9000/api/events/create

Input:

```
{
 "name": "Mani",
 "eventType": "BirthDay",
 "date": "2025-12-10 10:00:00",
 "guestCount": 120
}
```

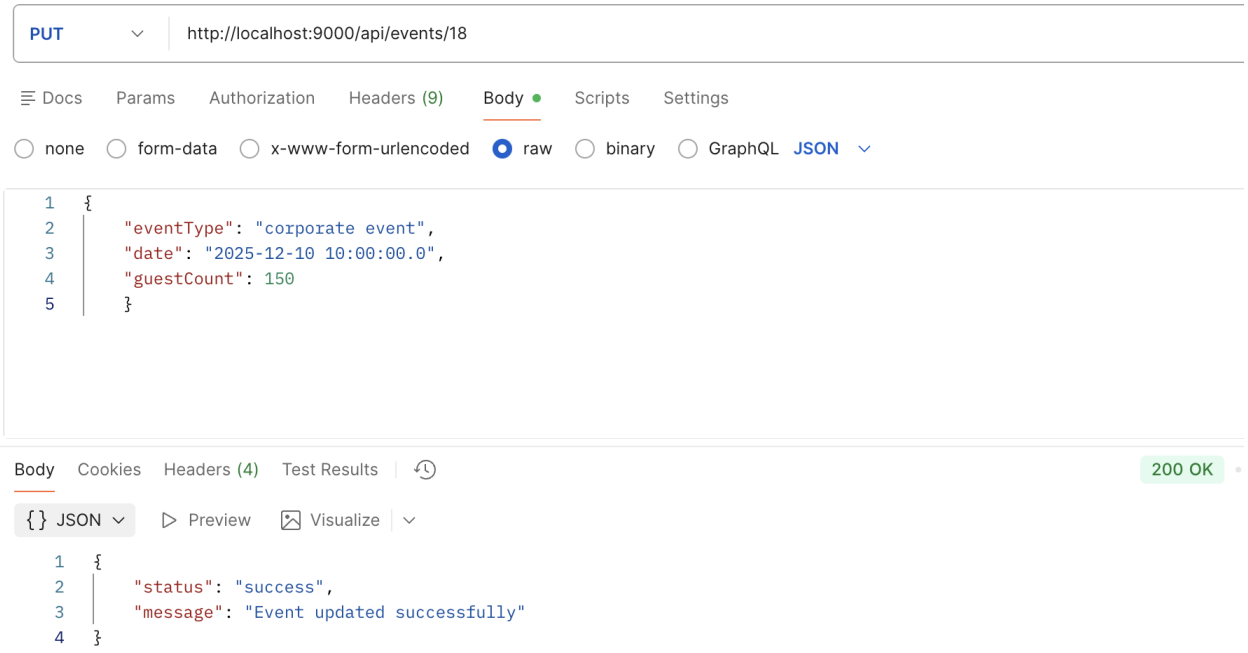
Output:

```
{
   "status": "success",
   "message": "Event created successfully",
   "data": {
       "id": 20,
       "name": "Mani",
       "eventType": "BirthDay",
       "date": "2025-12-10 10:00:00.0",
       "guestCount": 120,
       "createdBy": 6,
       "createdAt": "2025-11-23 01:45:11.078",
       "updatedAt": "2025-11-23 01:45:11.078"
   }
}
```



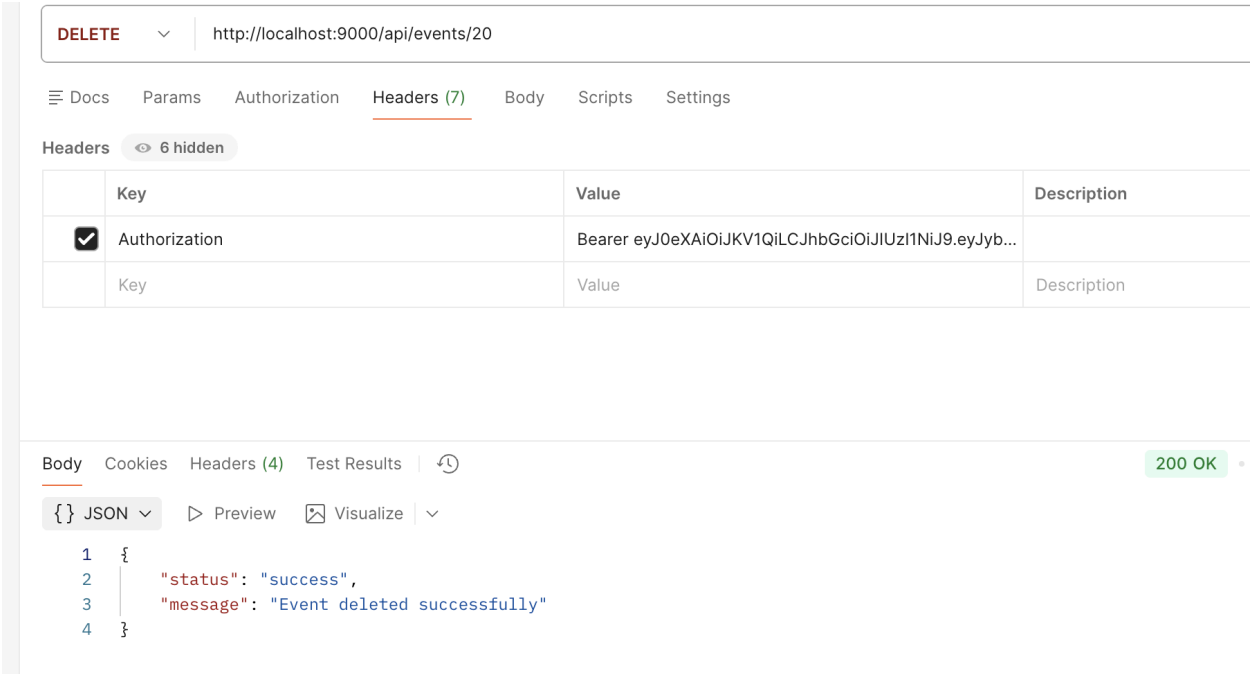
Update an Event

URL: PUT http://localhost:9000/api/events/18



Delete an Event

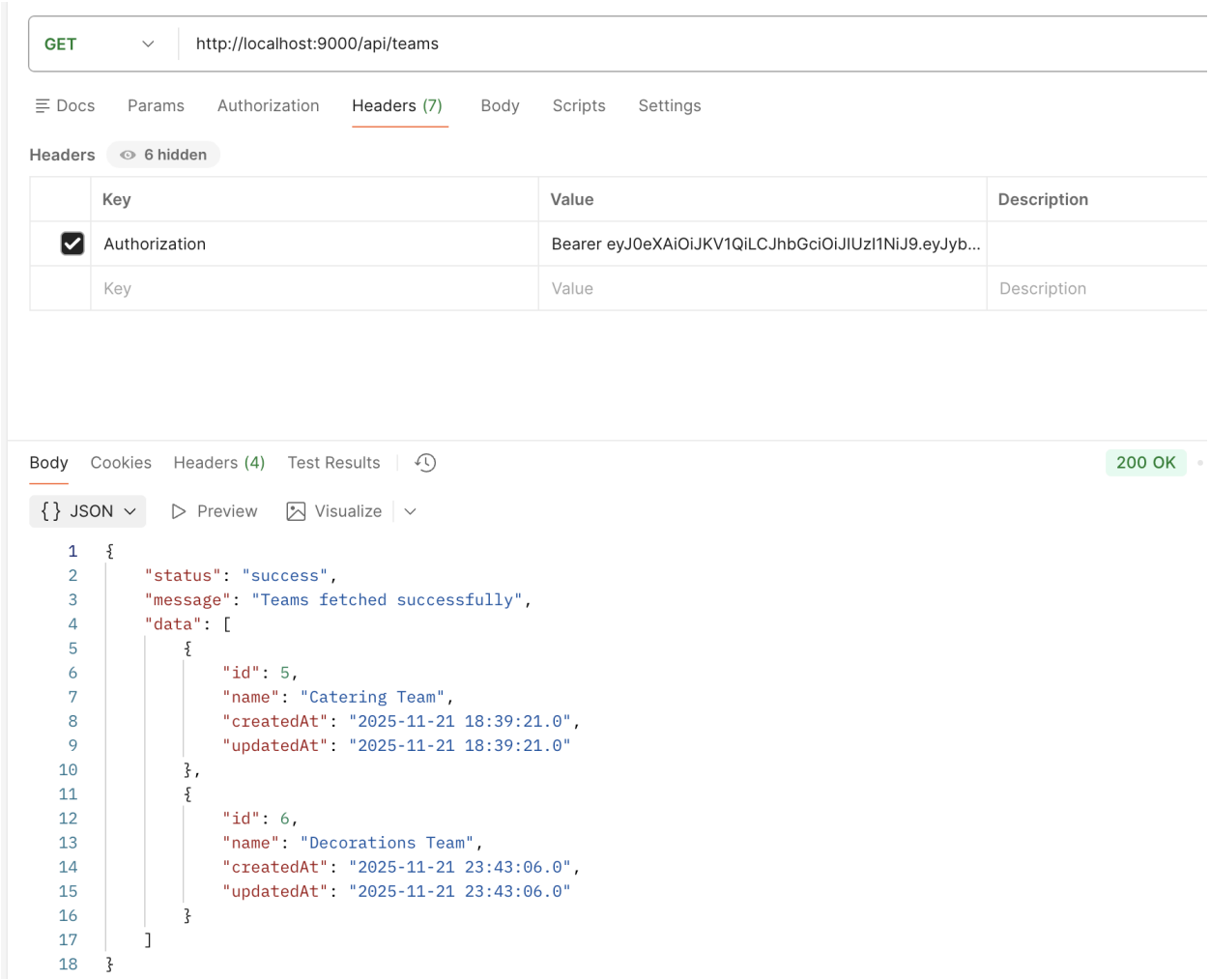
URL: DELETE http://localhost:9000/api/events/20



Team Management

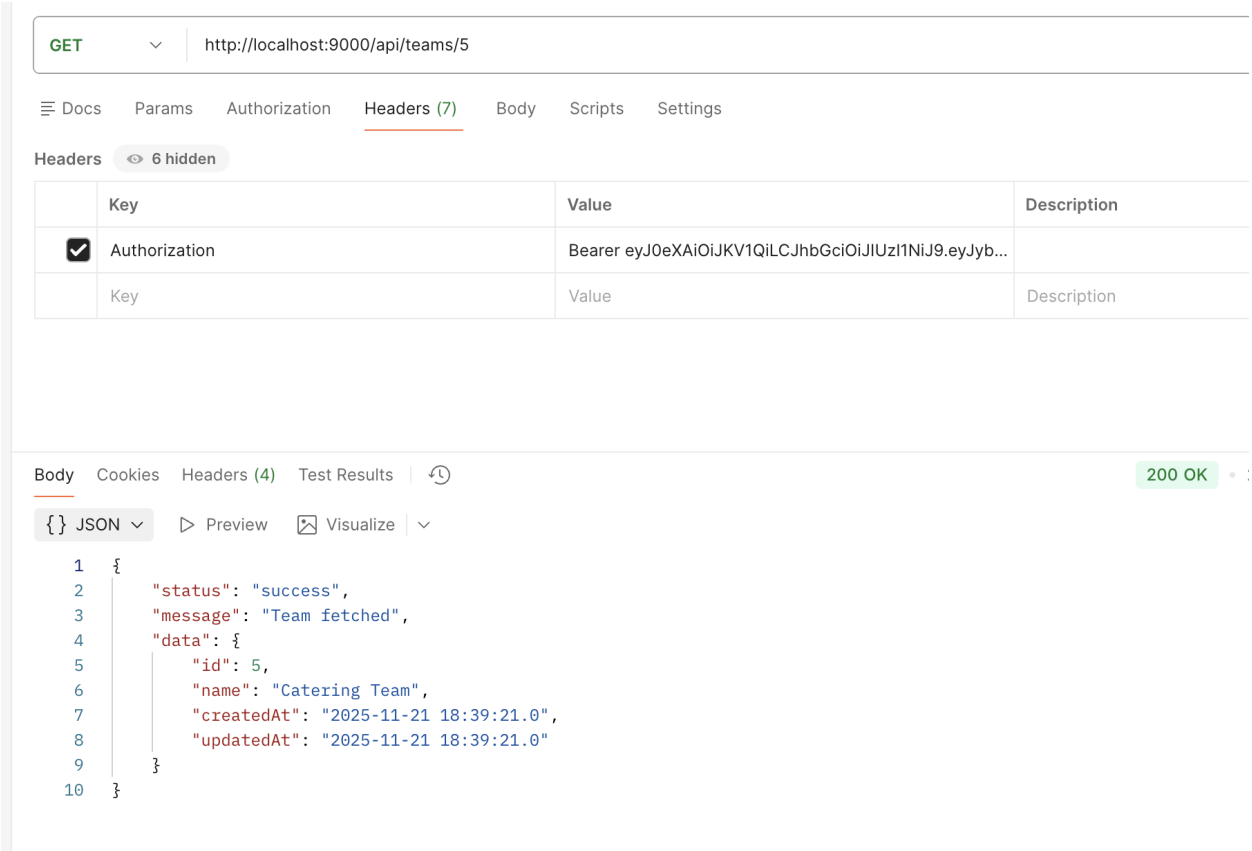
Get All Teams

URL: GET http://localhost:9000/api/teams



Get Team By ID

URL: GET http://localhost:9000/api/teams/5



Create a Team

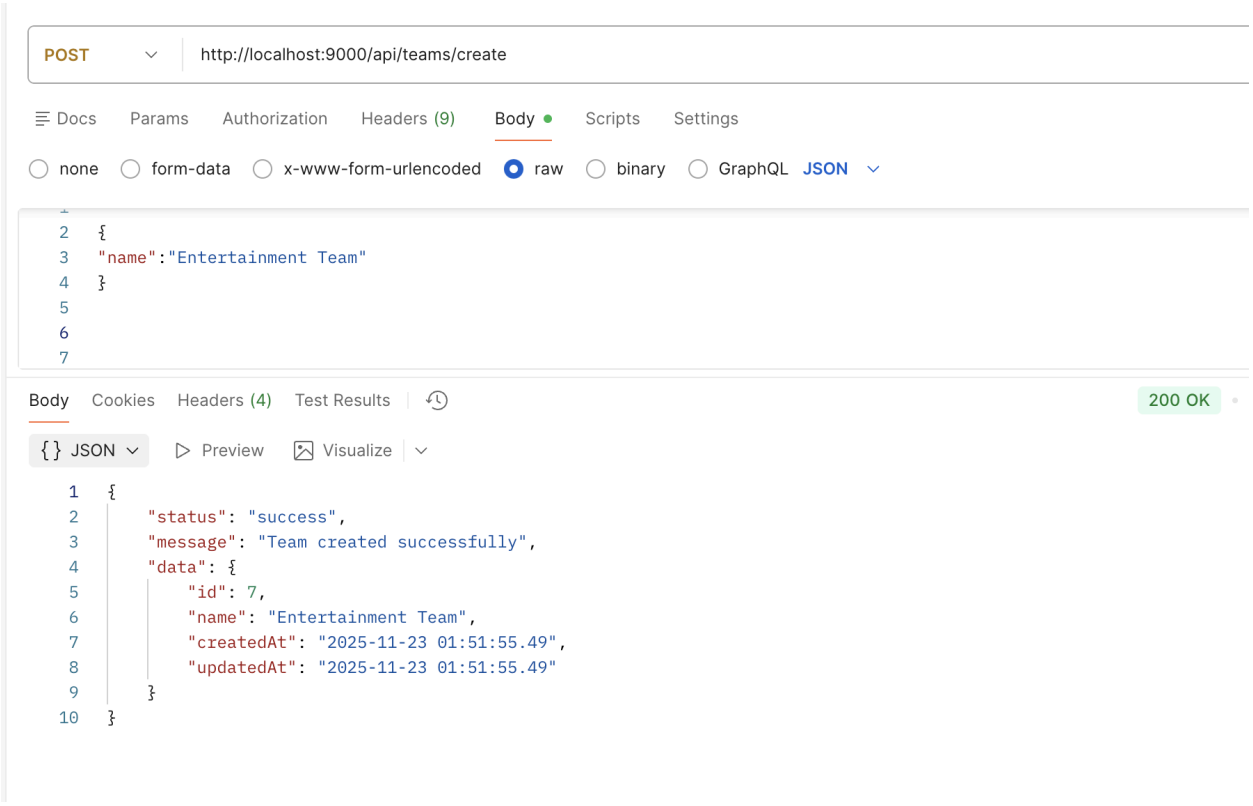
URL: POST http://localhost:9000/api/events/create

Input:

```
{
"name":"Entertainment Team"
}
```

Output:

```
{
    "status": "success",
    "message": "Team created successfully",
    "data": {
        "id": 7,
        "name": "Entertainment Team",
        "createdAt": "2025-11-23 01:51:55.49",
        "updatedAt": "2025-11-23 01:51:55.49"
    }
}
```



Update a Team

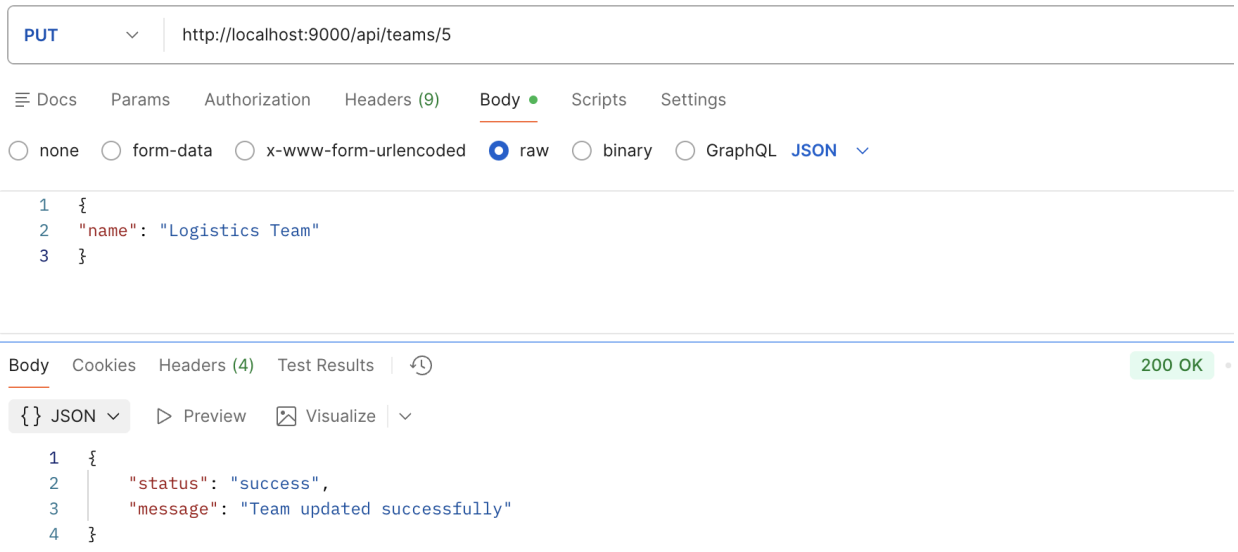
URL: PUT http://localhost:9000/api/teams/5

Input:

```
{
"name": "Logistics Team"
}
```

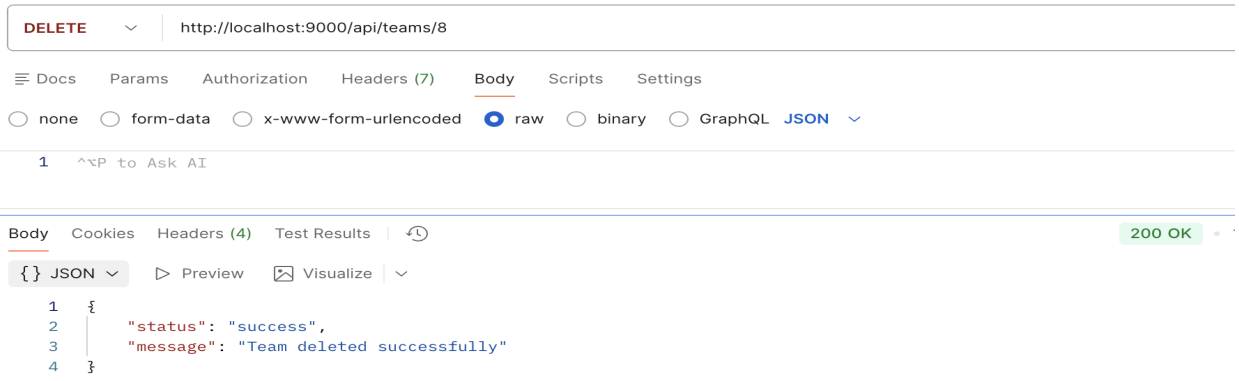
Output:

```
{
   "status": "success",
   "message": "Team updated successfully"
}
```



Delete a Team

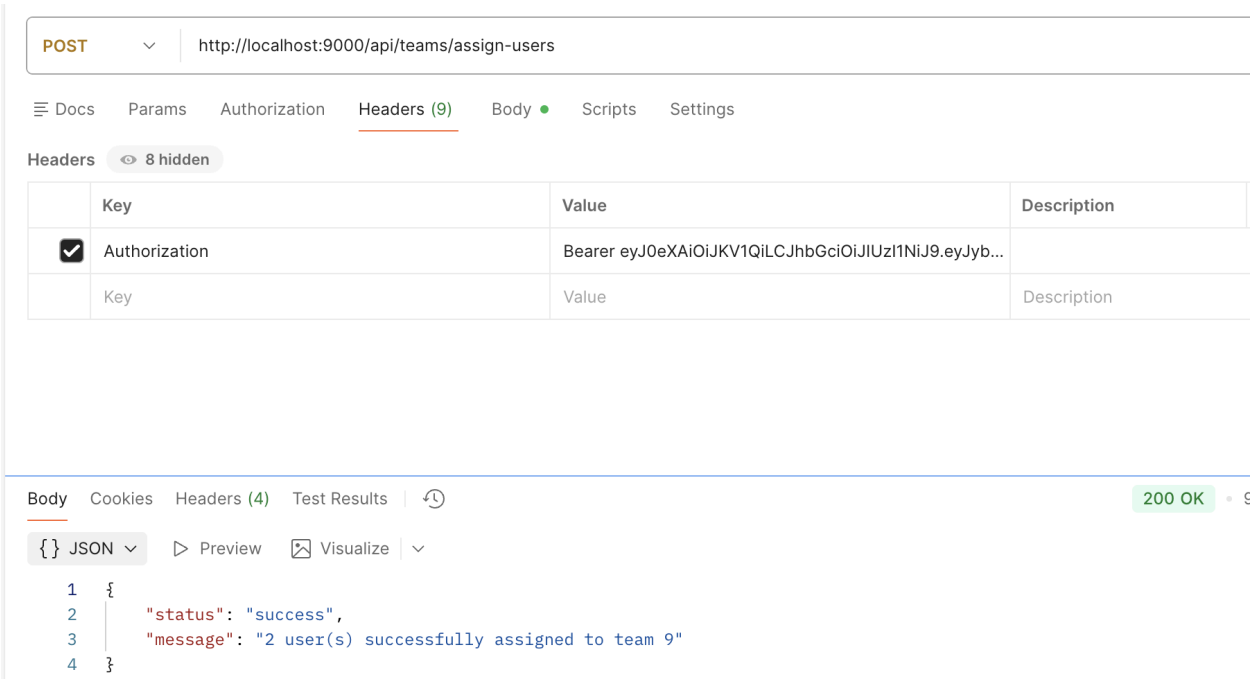
URL: DELETE http://localhost:9000/api/events/20



Team Assignment

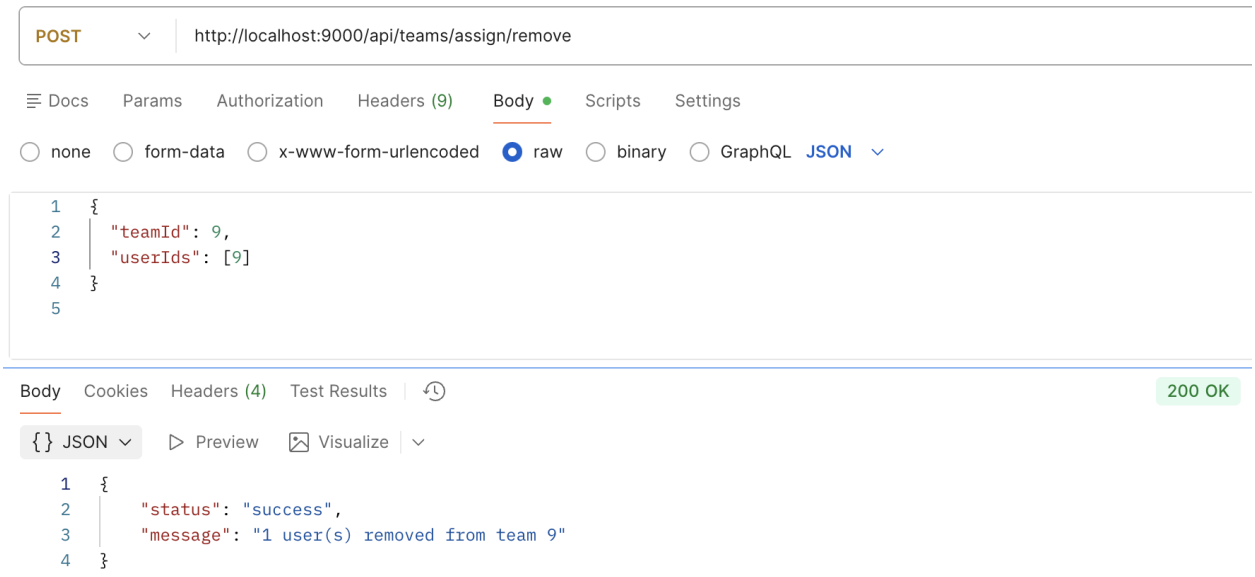
Team assignment

URL: POST http://localhost:9000/api/teams/assign-users



Team Unassign

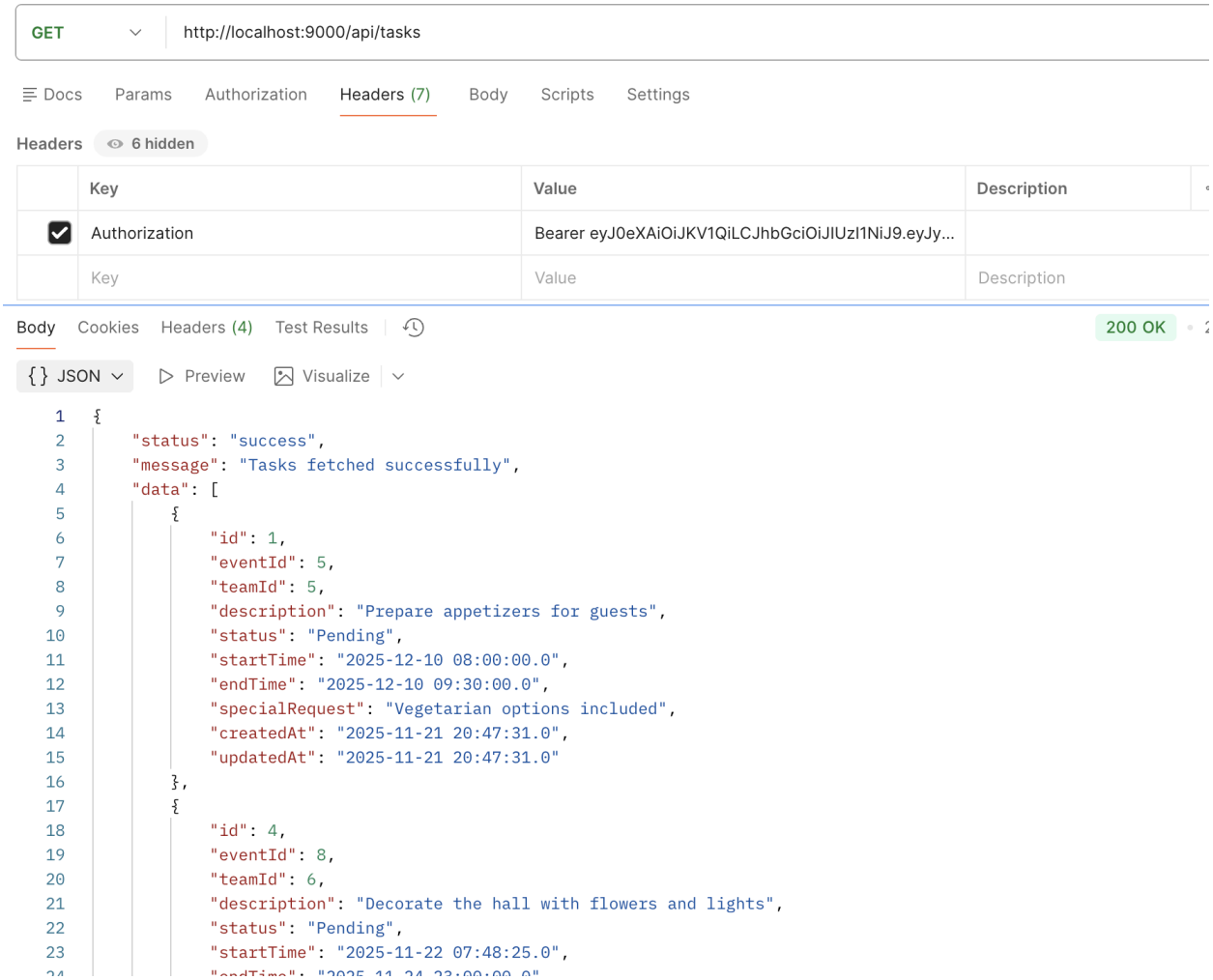
URL: POST http://localhost:9000/api/teams/assign/remove



Task Management

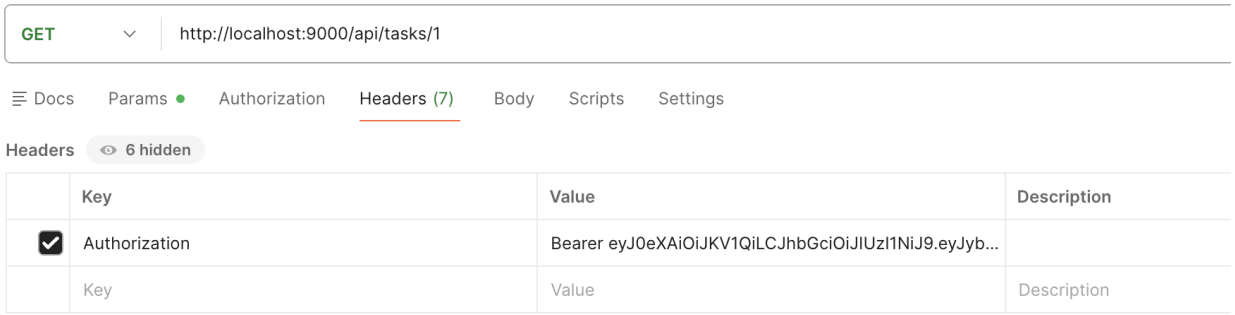
Get All Tasks

URL: GET http://localhost:9000/api/tasks



Get Task By ID

URL: GET http://localhost:9000/api/tasks/1



Create a Task

URL: POST http://localhost:9000/api/tasks/create

Input:

```
{
 "eventId": 18,
 "teamId": 9,
 "description": "Prepare and serve catering for the Annual Party",
 "status": "Pending",
 "startTime": "2025-11-20 22:05:00",
 "endTime": "2025-11-20 23:00:00",
```

```
        "specialRequest": "Include vegetarian and non-vegetarian options, setup
buffet table",
        "createdAt": "2025-11-20 22:00:00",
        "updatedAt": "2025-11-20 22:00:00"
       }
```


Output:


```
       {
          "status": "success",
          "message": "Task created successfully",
          "data": {
              "id": 8,
              "eventId": 18,
              "teamId": 9,
              "description": "Prepare and serve catering for the Annual Party",
              "status": "Pending",
              "startTime": "2025-11-20 22:05:00.0",
              "endTime": "2025-11-20 23:00:00.0",
              "specialRequest": "Include vegetarian and non-vegetarian options,
       setup buffet table",
              "reminderSent": false,
              "eventDayAlertSent": false,
              "createdAt": "2025-11-23 03:23:26.799",
              "updatedAt": "2025-11-23 03:23:26.799"
          }
       }
```

POST http://localhost:9000/api/tasks/create

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON


```
{
  "eventId": 18,
  "teamId": 9,
  "description": "Prepare and serve catering for the Annual Party",
  "status": "Pending",
  "startTime": "2025-11-20 22:05:00",
  "endTime": "2025-11-20 23:00:00",
  "specialRequest": "Include vegetarian and non-vegetarian options, setup buffet table",
  "createdAt": "2025-11-20 22:00:00",
  "updatedAt": "2025-11-20 22:00:00"
}
```


Body Cookies Headers (4) Test Results 200 OK

{} JSON Preview Visualize


```
{
    "status": "success",
    "message": "Task created successfully",
    "data": {
        "id": 5,
        "eventId": 18,
        "teamId": 9,
        "description": "Prepare and serve catering for the Annual Party",
        "status": "Pending",
        "startTime": "2025-11-20 22:05:00.0",
        "endTime": "2025-11-20 23:00:00.0",
        "specialRequest": "Include vegetarian and non-vegetarian options, setup buffet table",
        "reminderSent": false,
        "eventDayAlertSent": false,
        "createdAt": "2025-11-23 03:14:10.838",
        "updatedAt": "2025-11-23 03:14:10.838"
    }
}
```


POST http://localhost:9000/api/tasks/create

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

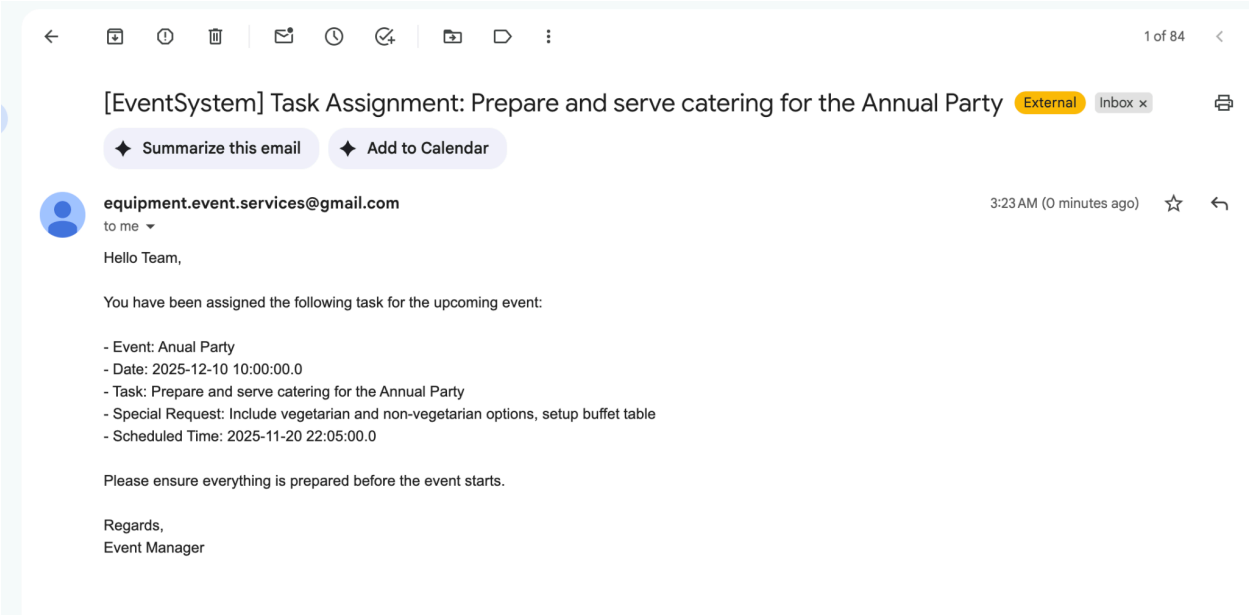

```
{
    "eventId": 18,
    "teamId": 9,
    "description": "Prepare and serve catering for the Annual Party",
    "status": "Pending",
    "startTime": "2025-12-09 12:00:00",
    "endTime": "2025-12-09 18:00:00",
    "specialRequest": "Include vegetarian and non-vegetarian options, setup buffet table",
    "createdAt": "2025-11-23 10:15:00",
    "updatedAt": "2025-11-23 10:15:00"
}
```


Body Cookies Headers (4) Test Results 200 OK

{} JSON Preview Visualize


```
{
    "status": "fail",
    "message": "Task already exists"
}
```

After task allocation, the task is created and notifications are sent to all assigned team members.



1 of 84

[EventSystem] Task Assignment: Prepare and serve catering for the Annual Party External Inbox ×

Summarize this email Add to Calendar

equipment.event.services@gmail.com
to me

3:23 AM (0 minutes ago)

Hello Team,

You have been assigned the following task for the upcoming event:

- Event: Anual Party
- Date: 2025-12-10 10:00:00.0
- Task: Prepare and serve catering for the Annual Party
- Special Request: Include vegetarian and non-vegetarian options, setup buffet table
- Scheduled Time: 2025-11-20 22:05:00.0

Please ensure everything is prepared before the event starts.

Regards,
Event Manager

Update a Task

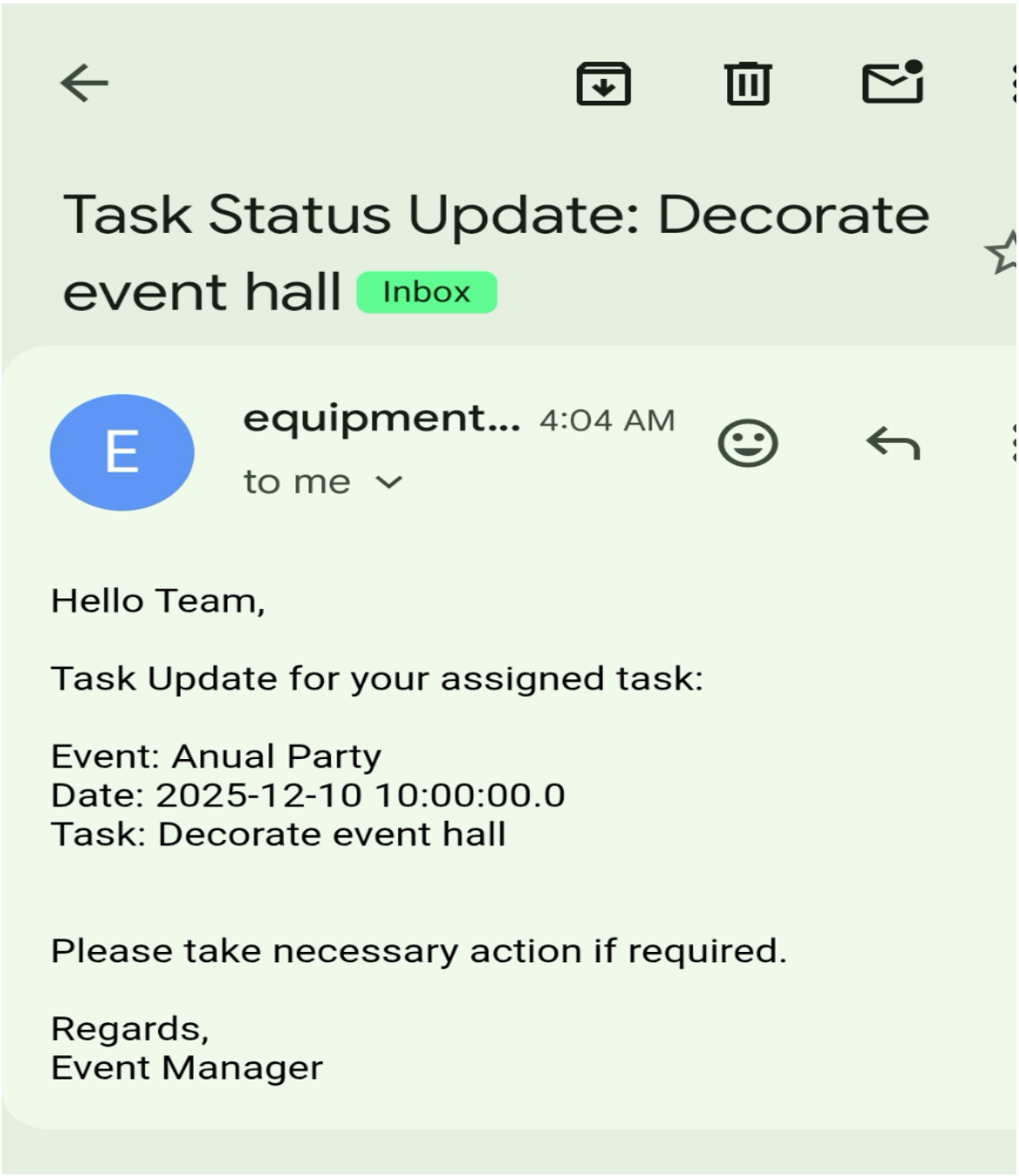
URL: PUT http://localhost:9000/api/tasks/10

Input:

```
{
   "status": "InProgress",
   "startTime": "2025-12-09 09:00:00"
}
```

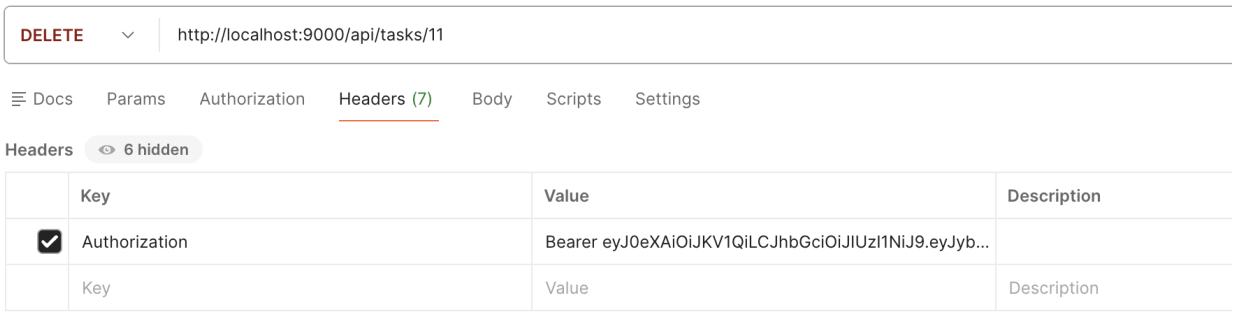
Output:

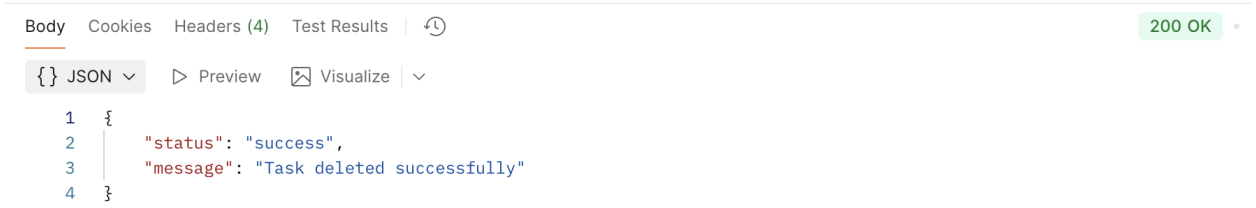
```
{
   "status": "success",
   "message": "Task updated successfully"
}
```


Task Status Update: Decorate event hall
Inbox
E
equipment... 4:04 AM
to me
Hello Team,
Task Update for your assigned task:
Event: Anual Party
Date: 2025-12-10 10:00:00.0
Task: Decorate event hall
Please take necessary action if required.
Regards,
Event Manager

Delete Task

URL: DELETE http://localhost:9000/api/tasks/11





Notification Service

- No public REST endpoints.
- Consumes Kafka events such as:
  - TaskAssigned
  - ReminderScheduled
  - IssueReported
  - EventDayAlertSends email/SMS notifications to service teams and event manager.

8. Security

Authentication

- JWT-based authentication for the event manager.

Authorization

Only one role:

- **Event Manager:** Full access to event and task management features.

Transport Layer Security

- All communication over HTTPS.

9. Key Design Decisions

- **Microservices Architecture:** Independent scalability of event management and notification workloads.
- **Kafka Messaging:** Guarantees reliable reminders even under network load.
- **Akka Actors:** Enables high-performance asynchronous reminders and real-time alerts.
- **Dockerization:** Simplifies deployment while keeping all environments consistent.

10. Conclusion

This architecture provides a scalable and reliable solution for coordinating multiple event service teams. The system ensures timely task assignment, continuous progress tracking, and automated reminders, allowing the event manager to maintain complete control over event preparation. With modular microservices and robust messaging, it is designed for future expansion and real-world efficiency.